

Technion – Israel Institute of Technology
Henry and Marilyn Taub Faculty of Computer Science

02360340 – Project in Computer Communications

NFT Copymint Detection

Replacing a Broken Signal in an
NFT Copymint Detector

Final Project Report

Team: Rabea Yassin, Ahmad Nassar
Instructor: Eran Tavor
Semester: Spring 2026

Abstract

A *copymint* takes an existing NFT image, edits it slightly, and mints it as new. Kotzer et al. detect copymints with four perceptual image hashes (aHash, pHash, hsvHash and sHash), each searched in its own BK-tree, and call an image a copy when at least two of the four agree. Our project set out to make that detector cheaper to run. While reimplementing it we found that sHash, the hash meant to survive cropping, does not define a distance in the mathematical sense: comparing two images gives a different answer depending on which image comes first, and the triangle inequality does not hold. A BK-tree depends on both properties, so a search can silently skip a branch that holds a real copy.

We therefore replaced sHash with ORB feature matching (keypoints, descriptor matching and RANSAC geometric verification) and kept everything else in the detector unchanged. On the geometric edits sHash was built for, ORB reaches F1 90.6 against 76.4 for sHash (+14.2), even though sHash was given a threshold chosen on the test set and ORB’s was chosen on training data. Held at equal precision (91.2%), sHash catches 27.2% of copies and ORB 90.0%. In the full four-signal detector, with the paper’s own thresholds left untouched, swapping only sHash for ORB raises F1 from 77.5 to 86.4 (+8.9): recall rises from 64.1% to 77.2% while precision stays at 98.1%. When the paper’s thresholds are re-tuned to our data the gain is 81.7 to 87.5 (+5.8).

The improvement has a cost that we report as a finding. The paper stores its four hashes, about 56 bytes, inside each mint transaction (300–500 bytes). ORB needs about 14.5 KB per image, roughly 36 transactions. Shrinking it removes the advantage before it fits: at 32 landmarks (1 KB, still 2.6 transactions) ORB only ties sHash.

Finally, we built an *edit classifier* that predicts from the query image alone which kind of edit was applied, so the detector could stop trusting signals known to fail on that edit. The classifier is accurate (100% on the “detail destroyed” decision, 97.5% F1 on “colour changed”), yet it does not improve the detector (F1 87.4 vs 87.5). A signal that breaks goes silent rather than voting wrongly, so ignoring it changes nothing. A multi-edit test then showed that the one situation where the classifier could help is not observable with our cues: pixelation erases the colour trace the classifier relies on.

Contents

Abstract	i
1 Introduction	1
1.1 General description	1
1.2 The starting point	1
1.3 What this report covers	1
2 Background	2
2.1 Perceptual image hashes	2
2.2 The paper’s detector	2
2.3 BK-trees need a metric	3
2.4 ORB, LSH and RANSAC	3
3 Goals and Timetable	3
3.1 Original goal	3
3.2 How the goals changed	4
3.3 Timetable	4
4 Architecture	5
4.1 The detector we evaluate	5
4.2 The ORB signal in three stages	5
4.3 The edit classifier	5
5 Dataset and Evaluation Method	6
5.1 Images and generated copies	6
5.2 Train/test split	6
5.3 The authors’ reference set	7
5.4 Pairwise evaluation	7
5.5 Metrics and operating points	7
5.6 Retiring an earlier ORB baseline	7
5.7 Reproducing the original hashes	8
6 Finding 1: sHash Cannot Be Searched Correctly	8
6.1 How sHash compares two images	8
6.2 It is not symmetric	8
6.3 The triangle inequality fails	9
6.4 Why this loses real copies	9
7 Results: ORB in Place of sHash	9
7.1 Result 1: head to head, on the edits sHash was built for	9

7.2	Result 2: recall by kind of edit	10
7.3	Result 3: inside the whole detector	11
8	Finding 2: What ORB Costs in Storage	12
9	Finding 3: The Edit Classifier Does Not Improve the Detector	13
9.1	The classifier is accurate	13
9.2	But broken signals are silent, not wrong	14
9.3	Measured end to end	14
9.4	The classifier is specific to our generator	15
9.5	The vote rule depends on how common copies are	15
9.6	Phase E: two edits on one image	16
10	Discussion	16
10.1	What we learned	16
10.2	Difficulties	17
11	Limitations and Future Work	17
12	Conclusion	18
	References	19
A	Setup and Run Guide	20
A.1	Environment	20
A.2	Data	20
A.3	Scoring (cached, run once per split)	20
A.4	Training and evaluation	20
A.5	C11 hash suite	21
B	Operating Points	21
C	Data Notes	21

List of Figures

1	The paper’s 2-Minimal detector. Four hashes, each searched in its own BK-tree; a copy needs two votes. Our project replaces the highlighted sHash.	2
2	The detector we evaluate: the paper’s design with ORB in place of sHash. The dashed edit classifier is evaluated separately and is not part of the headline results.	5
3	Real output of our ORB signal. Each green line is a match that RANSAC kept because it agrees with one geometric transformation. Hundreds of independent matches agreeing on one transformation does not happen by chance.	6
4	The edit categories in our generated dataset: the six edit types from the paper, plus exact copies as a control. Non-matching pairs serve as negatives.	7
5	The sHash comparison is one-way. B’s extra piece costs nothing when A is compared to B, and a lot when B is compared to A.	9
6	A BK-tree search missing a real copy because the sHash comparison breaks the triangle inequality.	9
7	Result 1. Left: F1 on the geometric subset, with the flag-everything level marked. Right: recall when both signals are held at 91.2% precision.	10
8	Result 2. Share of each edit’s copies that ORB catches (inliers > 16). *Our flip/rotate category always includes a rotation; pure mirrors alone are caught 100%.	11
9	Result 3. F1 of the full two-of-four detector with sHash and with ORB, at the paper’s own thresholds, at thresholds re-tuned to our data, and on the authors’ reference set.	12
10	ORB accuracy on the geometric subset against bytes stored per image. The shaded band is one mint transaction. No point on the ORB curve lies inside it.	13
11	F1 of the ORB detector under the two vote rules, reweighted to an assumed share of copies. The paper’s two-of-four rule wins wherever copies are uncommon.	15

List of Tables

1	Project stages, in order.	4
2	ORB against sHash on the geometric subset (3,600 copies, 2,400 non-copies).	10
3	The full detector before and after the swap. ORB votes at > 23 inliers in both settings.	11
4	Measured storage per image, on 60 real originals.	13
5	Selected points of the size curve (geometric subset, best threshold per budget).	14
6	Edit classifier decisions on the held-out test split.	14
7	Per-signal reliability map (%). “Caught” is the share of that edit’s copies the signal votes for; “wrong” is how often it votes for a deliberately wrong original.	14
8	Phase E results. Precision is about equal across detectors, so recall is the fair comparison.	16
9	Thresholds used in this report. Hash thresholds are Hamming distances; sHash’s is its averaged one-way distance.	21

1 Introduction

1.1 General description

Non-fungible tokens (NFTs) are blockchain records that point to a digital asset, most often an image. The blockchain proves who owns a token and how it changed hands, but nothing stops a second party from downloading the image, changing it a little, and minting the result as a new token. Such copies are called *copymints*. They lower the value of the original, mislead buyers, and are common enough that marketplaces such as OpenSea publish policies against them [1].

Detecting a copymint is an image-similarity problem, not an integrity problem. A cryptographic hash such as SHA-256 is useless here by design: changing one pixel produces an unrelated digest. A *perceptual* hash has the opposite goal: images that look alike should produce hashes that are close, for example in Hamming distance. A single perceptual hash is not enough, though, because each one survives some edits and fails on others. A colour histogram survives rotation but not recolouring. A coarse brightness layout survives recolouring but not rotation.

1.2 The starting point

Our project extends the detector of Kotzer, Reviriego, Conde Diaz and Rottenstreich [1, 2]. It computes four perceptual hashes per image, aHash, pHash, hsvHash and sHash, stores each in its own BK-tree, and flags a query as a copy when at least two of the four find a match within their thresholds (the paper’s *2-Minimal* detector). The paper also proposes storing the four hashes inside every mint transaction, so that any validator can check a new mint without downloading images.

1.3 What this report covers

We set out to make that detector cheaper to run, using a classifier that predicts the kind of edit and skips searches that cannot help (Section 3). Reimplementing the detector changed the direction of the project. We found that sHash cannot be searched correctly in a BK-tree (Section 6), replaced it with ORB feature matching, and measured the replacement on its own, inside the full detector, and against its storage cost (Sections 7–8). We also built the edit classifier and measured why it does not improve accuracy (Section 9).

The main contributions are:

1. a from-scratch C11 reimplementation of the paper’s four hashes, bit-exact against the library that produced the paper’s numbers;
2. a demonstration that the sHash comparison is not a metric, and a concrete case in which a BK-tree search misses a real copy because of it;
3. an ORB-based replacement for sHash, measured against sHash on the edits sHash was designed for and inside the paper’s full detector;
4. a measurement of what the replacement costs in storage, at every descriptor budget;
5. an edit classifier, and an explanation, backed by experiments, of why it does not improve the detector.

How to read the numbers. Our test set is 82.6% copies. On such a set a detector that answers “copy” for everything already scores F1 90.5. A raw F1 can therefore flatter a permissive

detector, so throughout this report we give precision next to every recall and compare detectors at equal precision wherever possible.

2 Background

2.1 Perceptual image hashes

All four of the paper’s hashes come from Buchner’s `imagehash` library [3], which also produced the paper’s reported numbers. Three of them are fixed-width binary strings compared with the Hamming distance (the number of differing bits):

- **aHash** (64 bits): shrink to 8×8 grayscale and set each bit by whether the pixel is brighter than the mean. A coarse brightness layout.
- **pHash** (64 bits): shrink to 32×32 , take the discrete cosine transform, and threshold the 8×8 lowest frequencies at their median. Coarse shapes.
- **hsvHash** (42 bits): a global histogram of the image’s colours, with black, gray and saturated pixels binned by hue. The paper’s text describes a block-based HSV hash, but the `colorhash()` function that generated its numbers is global. We followed the code, since that is where the paper’s numbers came from.

sHash is different. It segments the image into its largest regions and computes a 64-bit dHash (a horizontal-gradient hash) of each region’s bounding box. One image therefore yields a *list* of hashes whose length depends on the image. The paper adds it specifically to survive cropping: if part of the image is cut away, the remaining large objects should still match.

2.2 The paper’s detector

Each hash votes independently: it votes “copy” when the query’s nearest stored image is within that hash’s threshold. The *2-Minimal* detector reports a copy when at least two of the four vote, which keeps false alarms low because no single hash has to be trusted on its own (Figure 1). Formally, with d_i the distance under hash i and t_i its threshold,

$$\text{copy}(q, r) = \left[\sum_{i=1}^4 \mathbb{1}[d_i(q, r) \leq t_i] \geq 2 \right].$$

The thresholds given in the paper are $\text{aHash} \leq 7$, $\text{pHash} \leq 15$, $\text{hsvHash} \leq 3$ and $\text{sHash} \leq 17$ [1].

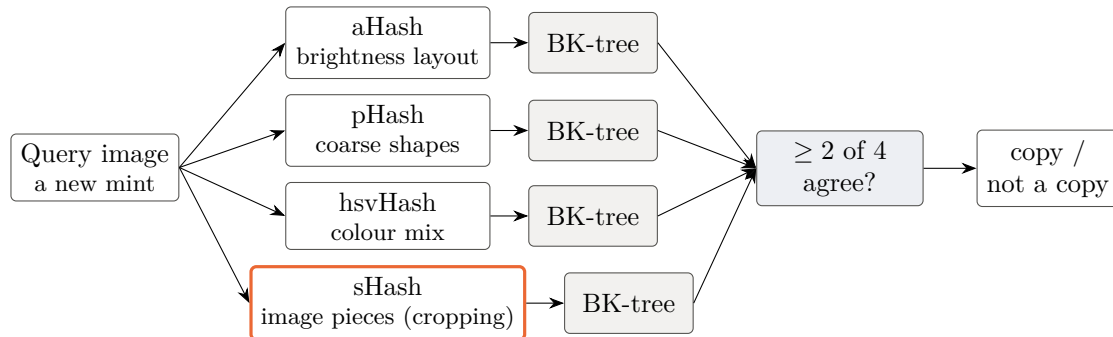


Figure 1: The paper’s 2-Minimal detector. Four hashes, each searched in its own BK-tree; a copy needs two votes. Our project replaces the highlighted sHash.

Storage on the blockchain. The paper proposes adding the four hashes as metadata fields of every mint transaction, so a validator can check a new mint “in a reasonable time” without fetching images. It notes that a transaction is usually 300–500 bytes and that each hash needs only a few bytes, so the overhead is low [1]. We use 400 bytes as a reference transaction. The four hashes take about 56 bytes: 24 fixed for aHash, pHash and hsvHash together, plus a median of 32 for sHash’s variable-length list (Section 8).

2.3 BK-trees need a metric

A BK-tree [4] stores one item per node and labels each child edge with the child’s distance from its parent. To find everything within distance t of a query q , the search computes $d = d(q, \text{node})$ and descends only into child edges labelled in $[d - t, d + t]$. This pruning is what makes the tree fast, and it is only safe when the distance is a *metric*: in particular it must be symmetric, $d(x, y) = d(y, x)$, and obey the triangle inequality, $d(x, z) \leq d(x, y) + d(y, z)$. The paper states this requirement itself. Hamming distance on fixed-width strings is a metric. A distance between variable-length lists is not automatically one.

2.4 ORB, LSH and RANSAC

ORB (Oriented FAST and Rotated BRIEF) [5] finds *keypoints*, corners and sharp junctions that can be found again after an edit, and describes the neighbourhood of each with a 256-bit (32-byte) binary descriptor. Each keypoint also gets an orientation, which is what lets matches survive rotation. Two images are compared by matching descriptors in Hamming distance. Many matches are coincidental, so RANSAC [6] searches for the single geometric transformation (here a homography) that the most matches agree on and discards the rest. The number of agreeing matches, the *inliers*, is the similarity score.

Because one image yields hundreds of descriptors rather than one string, ORB cannot be put in a BK-tree. The natural index for binary descriptors is locality-sensitive hashing (LSH) [7], which places similar descriptors in the same bucket so that only a short list of candidates needs RANSAC verification.

3 Goals and Timetable

3.1 Original goal

The original goal was **efficiency**: make the paper’s detector cheaper to run without changing its answers. The idea was an *edit classifier*. It would look at the query image first, predict what kind of edit had been applied, and run only the BK-tree searches that could still contribute a vote. For example, a pixelated image destroys the fine detail that a structure-based signal needs, so that signal’s search could be skipped.

The planned steps were:

1. reimplement the paper’s four hashes in C11, bit-exact against the reference library;
2. build a labelled dataset of NFT copies with known edits;
3. train the edit classifier;
4. benchmark the classifier-guided detector against the original for speed.

3.2 How the goals changed

To build the classifier we had to understand exactly where each signal breaks, and that work changed the project in three ways:

- **sHash cannot be searched correctly** (Section 6). Accelerating four BK-tree searches makes little sense when one of the four trees can miss real copies.
- **A C11-versus-Python speed comparison would not be a fair claim.** Our instructor pointed out that comparing a C11 implementation against the paper’s Python mixes language effects with algorithmic ones.
- **A better target existed.** The paper’s own future-work section points at its geometric blind spots, and feature matching addresses exactly those.

The revised goals were therefore about **accuracy**:

1. replace sHash with a signal that does its job, cropping and rotation, and can be indexed soundly;
2. measure the replacement fairly: against sHash on its own edits, inside the paper’s unchanged detector, and against its storage cost;
3. re-aim the edit classifier from speed to accuracy (stop trusting a signal on the edits known to break it) and measure whether that helps.

The C11 hash suite was kept as validated groundwork. The rest of the project was done in Python.

3.3 Timetable

Stage	Work	Outcome
1. Learning	The paper, perceptual hashing, BK-trees, the <code>imagehash</code> code	Project plan
2. Reproduction	C11 hash suite; differential tests on 600 images and the authors’ data	Bit-exact hashes; the sHash finding
3. Pivot	sHash analysis; review of a first ORB pipeline	Old ORB numbers retired; pairwise evaluation adopted
4. Phase B: ORB	Pairwise ORB signal, threshold tuning, comparison with sHash, storage curve	Results 1–2; storage finding
5. Phase C: edit classifier	93 features, Random Forest, per-signal reliability map	Accurate classifier
6. Phase D: full detector	Three-way comparison, comparison at the paper’s own thresholds, cross-generator check	Result 3; classifier adds ≈ 0
7. Phase E: multi-edit test	Two-edit generator, independent classifier heads, four-way comparison	The colour trace is erased by pixelation
8. Write-up	Presentation, final report, poster	This document

Table 1: Project stages, in order.

4 Architecture

4.1 The detector we evaluate

Our detector keeps the paper’s design intact and changes exactly one component: ORB replaces sHash as the fourth signal. The other three hashes, their thresholds and the two-vote rule are untouched, so any change in accuracy can be attributed to the swap (Figure 2). The edit classifier is an optional addition, drawn dashed. It reads only the query and tells the vote which signals to ignore. Section 9 shows that it does not change the result, so every headline number in this report is for the detector without it.

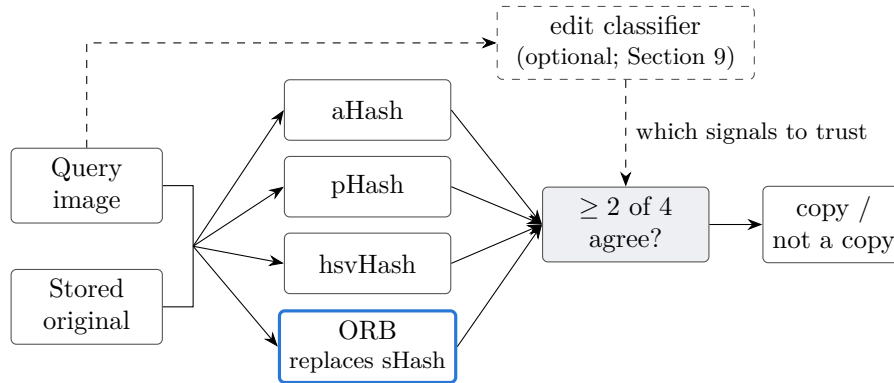


Figure 2: The detector we evaluate: the paper’s design with ORB in place of sHash. The dashed edit classifier is evaluated separately and is not part of the headline results.

4.2 The ORB signal in three stages

1. **Find landmarks and match them.** ORB finds a few hundred keypoints per image and describes each one; descriptors are matched between the two images by Hamming distance with cross-checking (Figure 3).
2. **Search a whole collection.** In deployment a new mint must be compared against every stored image. LSH buckets similar descriptors so that only a short list of candidates reaches stage 3. *Every accuracy number in this report compares two images directly, with no index involved*, so none of the accuracy comes from this stage.
3. **Throw out coincidences.** RANSAC finds the one homography that most matches agree on. The number of agreeing matches (inliers) is the score. A pair is a copy when the score exceeds a threshold: more than 16 in the signal study of Section 7, and more than 23 inside the full detector, where every signal is held to the same false-alarm budget (Section 5).

Resolution normalisation. CryptoPunks are 24×24 pixel art, while ORB’s default patch and border sizes are 31 pixels. On a native punk ORB finds no keypoints at all. Every image is therefore scaled to a common 256-pixel working edge before ORB, using nearest-neighbour interpolation when enlarging so that pixel-art edges stay sharp (225 keypoints on average, against 186 with Lanczos and 0 at native size).

4.3 The edit classifier

The edit classifier receives *only the query image*; at check time there is no original to compare against. It computes 93 single-image features (48 of them colour-histogram bins; the rest mea-

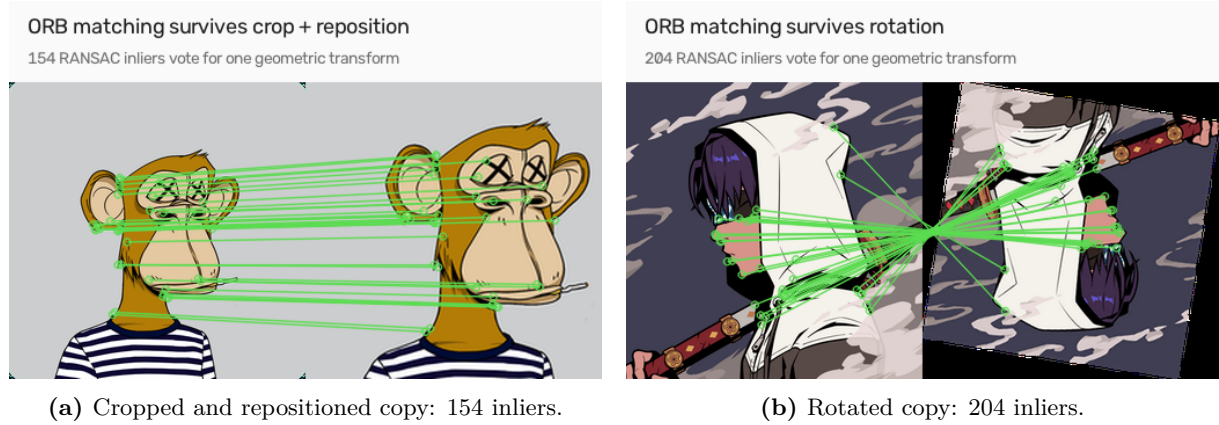


Figure 3: Real output of our ORB signal. Each green line is a match that RANSAC kept because it agrees with one geometric transformation. Hundreds of independent matches agreeing on one transformation does not happen by chance.

sure palette size, flat-neighbour fraction, transparency at the corners, sharpness and histogram “combing”) and feeds them to a Random Forest [8, 9] that outputs a probability for each of the eight edit labels. The detector uses two summed probabilities rather than the top label:

$$\begin{aligned}
 P(\text{detail destroyed}) &= P(\text{pixelated}) && \Rightarrow \text{stop trusting ORB} \\
 P(\text{colour changed}) &= P(\text{colour swap}) + P(\text{background change}) && \Rightarrow \text{stop trusting hsvHash}
 \end{aligned}$$

When signals are distrusted, the vote is taken among the remaining ones (“two of the trusted”, falling to one of two when only two remain). A static fallback keeps the original two-of-four verdict whenever it fires, so the classifier can only add detections.

5 Dataset and Evaluation Method

5.1 Images and generated copies

The paper’s evaluation data is not public, so we built our own. The base corpus is 3,000 real NFT images from a public Kaggle collection [11]: 1,000 each from Azuki, Bored Ape Yacht Club (BAYC) and CryptoPunks. Azuki and BAYC are ordinary artwork, while CryptoPunks are 24×24 pixel art. A separate 10,000-image CryptoPunks set [12] was kept aside for stress tests and not mixed in, so that one collection does not dominate.

The images carry no edit labels, so we generated copies ourselves, following the six edit types the paper takes from OpenSea’s copymint examples (Figure 4): flip, rotate or mirror; crop, resize or reposition; a text, logo or emoji overlay; a background-colour change; pixelation; and a colour swap or saturation change. We also generated exact copies as a control and non-matching pairs as negatives. Each edit is applied with three random variations per base image, using PIL.

One labelling caveat. Our generator’s `flip_rotate_mirror` edit applies an optional mirror and then *always* a rotation of 1–359 degrees. The category therefore contains no pure mirrors, and every result for it describes rotation. We separate the two in Section 7.

5.2 Train/test split

The split is made **by base image, before any edit is applied**, so no image’s copies appear in both training and testing. We use 80% for training and 20% for testing: 2,400 training bases

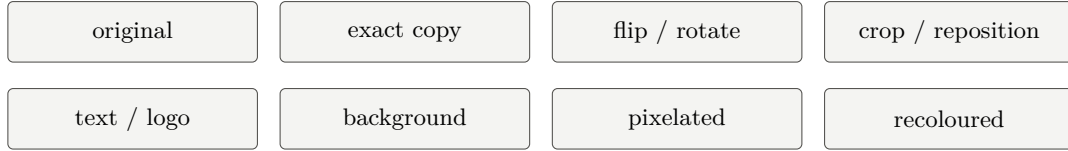


Figure 4: The edit categories in our generated dataset: the six edit types from the paper, plus exact copies as a control. Non-matching pairs serve as negatives.

(48,000 generated images, 55,200 labelled pairs) and 600 test bases (12,000 images, 13,800 pairs). The paper uses the opposite split (20/80), because it tunes only a few scalar thresholds. We train a Random Forest over 93 features and eight labels, which benefits from more training data. The test split contains **11,400 copies and 2,400 non-copies**.

5.3 The authors’ reference set

The paper’s authors shared with us a small set of their own manipulated images: 202 manipulated CryptoPunks and 200 originals, with a 1,802-row file of labelled original–candidate pairs (202 copies and 1,600 manipulated non-copies) and their own precomputed hashes and distances. We use it for two things: to check that our reimplementations matches theirs, and as an independent test made by a different generator. The set was shared privately, so it stays local and is not redistributed.

5.4 Pairwise evaluation

Each test row is a pair (stored original, query) with a label saying whether the query is a copy of that original. Every signal is scored on the pair directly, with no gallery or index in the accuracy path. This matches how the dataset is labelled and how the hashes are evaluated. It also rules out the failure described in Section 5.6, where an index decided the result instead of the signal.

5.5 Metrics and operating points

We report precision $P = \frac{TP}{TP+FP}$, recall $R = \frac{TP}{TP+FN}$ and $F_1 = \frac{2PR}{P+R}$. Two rules keep these honest:

- **A recall always comes with its precision.** The single-edit test set is 82.6% copies, so “flag everything” scores F1 90.5. On the geometric subset used for the sHash comparison (60% copies) it scores 75.0. Detectors are compared at equal precision wherever possible.
- **Thresholds are chosen on training data and frozen for test.** The one exception is deliberate and works against us: in the head-to-head comparison sHash’s threshold is chosen on the test set itself.

Inside the full detector, every signal is placed at the same false-alarm budget, at most 10% of training non-copies flagged: aHash ≤ 7 , pHash ≤ 19 , hsvHash ≤ 3 , sHash ≤ 21 , ORB > 23 inliers. ORB’s > 16 cutoff from the signal study sits at about 14% false alarms, so using it inside the detector would give ORB a looser vote than the others. We also evaluate the detector at the paper’s own thresholds (Section 7).

5.6 Retiring an earlier ORB baseline

The project inherited a first ORB pipeline that reported F1 73.6% (precision 82.7%, recall 66.3%). Before building on it we checked it, and found that its gallery held 2,000 of the 3,000

originals: every CryptoPunk was missing. At 24×24 pixels ORB found no keypoints in any punk, and the indexing code skipped images without descriptors without a warning. The result was decided by which collection an image belonged to, not by whether it was a copy. A script that does no image matching at all, only checking whether a filename is in the gallery, reproduces all four cells of the reported confusion matrix exactly (30,210 TP, 15,390 FN, 6,306 FP, 3,294 TN). The threshold sweep was also flat (F1 73.58 to 73.52 across every setting), which confirmed that the ORB score was not deciding anything. We retired those numbers and rebuilt the evaluation pairwise, with the resolution normalisation of Section 4.

5.7 Reproducing the original hashes

Before changing the detector, we reimplemented the four hashes in C11, plus dHash which sHash uses internally. All five match Buchner’s `imagehash` exactly on 600 distinct images, including all 402 of the authors’ reference images, high-resolution NFTs, native CryptoPunks and synthetic edge cases. For sHash we matched the full ordered list of segment hashes, not only the final distance. Our sHash distance also reproduces all 1,802 distances in the authors’ reference file. Two details made exact parity hard. First, Pillow declares some variables as `float` but computes with `double` literals, so a natural all-float port differs on about 0.4% of pixels. Second, the paper’s description of hsvHash differs from the code that produced its numbers, as noted in Section 2.1.

6 Finding 1: sHash Cannot Be Searched Correctly

6.1 How sHash compares two images

In the paper’s storage design (its Fig. 11b) each sHash BK-tree node holds one image’s list of piece hashes. Comparing image A , with pieces $a_1, \dots, a_m$, to image B , with pieces $b_1, \dots, b_n$, works as follows: for each piece of A , find its closest piece in B , then average those closest distances:

$$d_s(A \rightarrow B) = \frac{1}{m} \sum_{i=1}^m \min_j d_H(a_i, b_j).$$

The averaging is not stated in the paper; it comes from the authors’ code and data, which our C11 reimplementations reproduces exactly. Only the pieces of the first image have to find a match, so the comparison runs one way.

6.2 It is not symmetric

Let image A have 3 pieces and image B the same 3 plus one more that matches nothing in A (Figure 5). With matching pieces 2 apart and the unmatched piece 34 from its nearest neighbour:

$$d_s(A \rightarrow B) = \frac{2+2+2}{3} = 2, \quad d_s(B \rightarrow A) = \frac{2+2+2+34}{4} = 10.$$

The same two images get two different answers. The data the authors shared with us shows this in practice. The paper’s text describes comparing the *new* image to the original, but their reported distances match the original→copy direction on all 1,802 pairs and the copy→original direction on only 240. With a one-way comparison, this is an easy detail to miss.

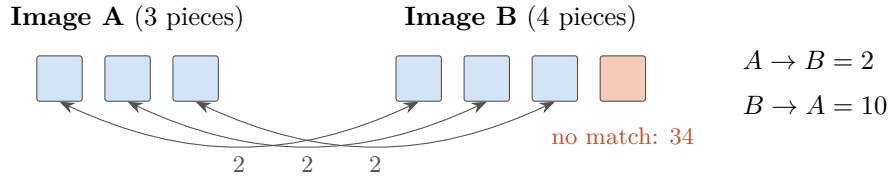


Figure 5: The sHash comparison is one-way. B’s extra piece costs nothing when A is compared to B, and a lot when B is compared to A.

6.3 The triangle inequality fails

Let A be an image of a cat, C an image of a dog, and B an image containing both. Take cat-to-cat and dog-to-dog pieces as 2 apart and cat-to-dog as 30 (illustrative values):

$$d(A, B) = 2, \quad d(B, C) = \frac{30+2}{2} = 16, \quad d(A, C) = 30 > 2 + 16 = 18.$$

The direct distance is longer than the detour through B , which a metric does not allow. A composite image sits close to two unrelated images at once.

6.4 Why this loses real copies

The tree’s pruning relies on exactly that inequality. Suppose the composite B is stored at a node, and a genuine copy X of a cat is stored below it. Because of B ’s dog, the stored edge is $d(B, X) = 16$. A cat query q arrives with $d(q, B) = 2$, and the true copy is only $d(q, X) = 2$ away. With threshold $t = 5$ the search descends only into edges in $[2 - 5, 2 + 5] = [-3, 7]$, so the edge labelled 16 is skipped (Figure 6). The search reports “no copy” and nothing looks wrong. A silently missed copy is the most costly error for this system.

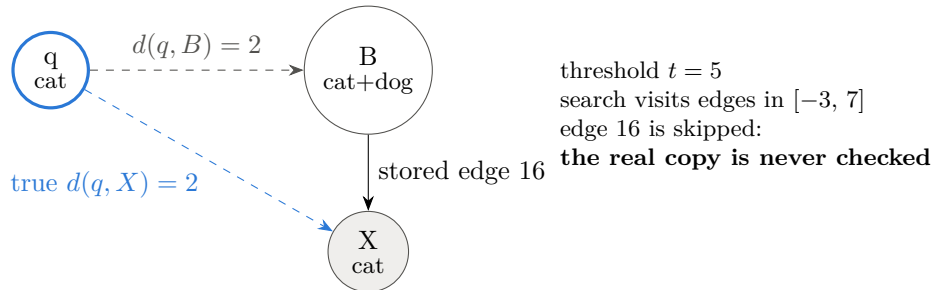


Figure 6: A BK-tree search missing a real copy because the sHash comparison breaks the triangle inequality.

We shared this analysis with the authors, who replied that it may help explain why the tree’s results are not perfect. Our claim is limited to what the code and the mathematics show: the sHash comparison is not a metric, so standard BK-tree pruning can miss a valid match. This changed the project. It made little sense to accelerate a search structure that can miss real copies, so we turned to replacing the signal.

7 Results: ORB in Place of sHash

7.1 Result 1: head to head, on the edits sHash was built for

sHash exists to survive cropping and other geometric edits, so the first test uses exactly those: the 3,600 test copies made by crop/resize/reposition and flip/rotate, plus the 2,400 non-copies,

60% copies in all. The comparison is deliberately tilted against ORB. ORB’s threshold (> 16 inliers) was chosen on training data. sHash was given its best possible threshold, chosen by sweeping *the test set itself* ($d_s \leq 28$), which no real deployment could do.

Signal	Threshold	Precision	Recall	F1
sHash (oracle threshold, chosen on test)	$d_s \leq 28$	66.5%	89.8%	76.4%
ORB (threshold chosen on train)	inliers > 16	91.2%	90.0%	90.6%
Flag everything	—	60.0%	100%	75.0%

Table 2: ORB against sHash on the geometric subset (3,600 copies, 2,400 non-copies).

ORB scores **+14.2 F1** on the subset sHash was designed for, with sHash holding the oracle advantage. The recall column looks like a tie (89.8% vs 90.0%), but it is not. sHash reaches its recall by flagging almost everything: **1,630 of the 2,400 non-copies** (68%), against roughly 310 (about 13%) for ORB. Its precision (66.5%) is barely above the 60% share of copies, and its F1 is only 1.4 points above flagging everything. Held at ORB’s precision of 91.2%, sHash catches **27.2%** of the copies against ORB’s **90.0%** (Figure 7). There is no threshold at which sHash is the better choice.

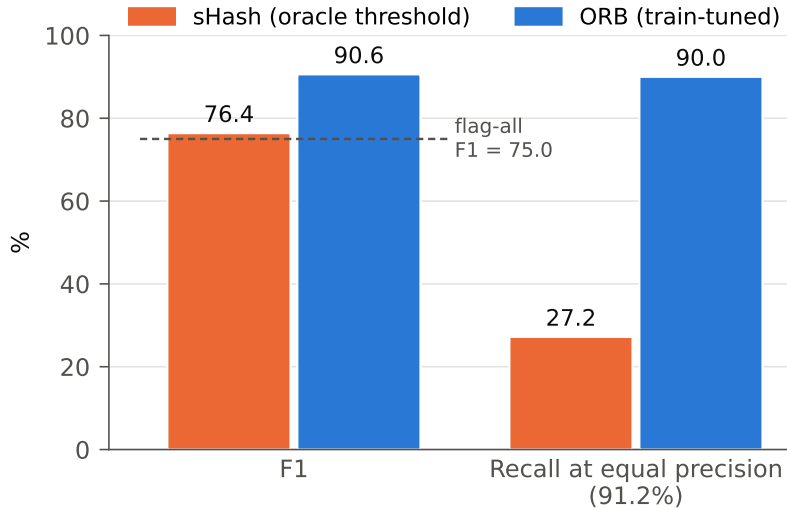


Figure 7: Result 1. Left: F1 on the geometric subset, with the flag-everything level marked. Right: recall when both signals are held at 91.2% precision.

7.2 Result 2: recall by kind of edit

Figure 8 breaks ORB’s recall down by edit type over the whole single-edit test set, at the same > 16 cutoff. At that cutoff ORB’s precision on the geometric subset is 91.2%, and it flags 13.0% of all non-copies.

- **It does the job it was brought in for.** Crop/reposition: 98.3%.
- **It also survives edits we did not aim it at.** A recoloured background (98.9%) and a pasted logo (100%) barely disturb the shapes ORB matches. ORB is better described as a *structure* signal than a geometric one: it works wherever local structure survives.
- **Flip/rotate (81.7%) is a rotation result.** Scoring the parts separately on 120 test originals shows pure mirrors at 100% for all three collections. The whole deficit is rotated

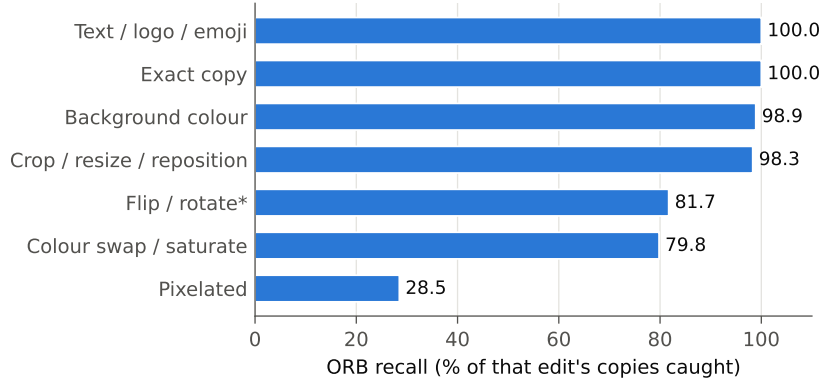


Figure 8: Result 2. Share of each edit’s copies that ORB catches (inliers > 16). *Our flip/rotate category always includes a rotation; pure mirrors alone are caught 100%.

CryptoPunks: rotation and resizing blur the hard pixel-art edges ORB relies on, and a 24×24 image has no detail to spare.

- **One clean failure: pixelation (28.5%).** Pixelation destroys the fine detail that corners are made of. The other signals still cover it: at operating points that flag under 10% of non-copies, pHash catches 65.5% of pixelated copies and hsvHash 79.3%.

CryptoPunks are the hard case. At a 9.9% false-alarm rate, ORB catches 100% of geometric copies for Azuki and BAYC but 68.5% for CryptoPunks. Enlarging a 24×24 image recovers keypoints but cannot create detail that was never there. CryptoPunks is also the collection the paper used for its main evaluation.

7.3 Result 3: inside the whole detector

The decisive test changes one signal and leaves everything else as the paper specifies it: the same three hashes, the same thresholds, the same two-of-four rule. On our full single-edit test set (11,400 copies, 2,400 non-copies):

Detector (2 of 4 agree)	Precision	Recall	F1
<i>Paper’s own thresholds, frozen</i>			
with sHash (the paper’s detector)	98.1%	64.1%	77.5%
with ORB in place of sHash	98.1%	77.2%	86.4%
<i>Thresholds re-tuned to our training data</i>			
with sHash	97.6%	70.3%	81.7%
with ORB in place of sHash	97.8%	79.2%	87.5%
Flag everything	82.6%	100%	90.5%

Table 3: The full detector before and after the swap. ORB votes at > 23 inliers in both settings.

Against the paper’s detector at its own thresholds, the swap raises F1 from 77.5 to 86.4, **+8.9**. Because precision is 98.1% in both rows, this is a clean comparison at equal precision: recall rises from 64.1% to 77.2%, about 1,500 more of the 11,400 copies caught at the same precision. Re-tuning the paper’s thresholds to our data helps the paper (77.5 to 81.7); the swap still wins, 81.7 to 87.5 (**+5.8**). ORB votes at > 23 inliers in both settings. The re-tuned baseline’s gain comes from the other hashes: pHash moves from ≤ 15 to ≤ 19 , and in the paper’s detector sHash from ≤ 17 to ≤ 21 .

The gain is located where Result 2 predicts. In the re-tuned detector, flip/rotate copies go from 14.1% to 58.4% caught and crop/reposition from 69.9% to 80.9%. Pixelated copies drop slightly, from 71.1% to 66.4%, because sHash sometimes still voted there and ORB does not.

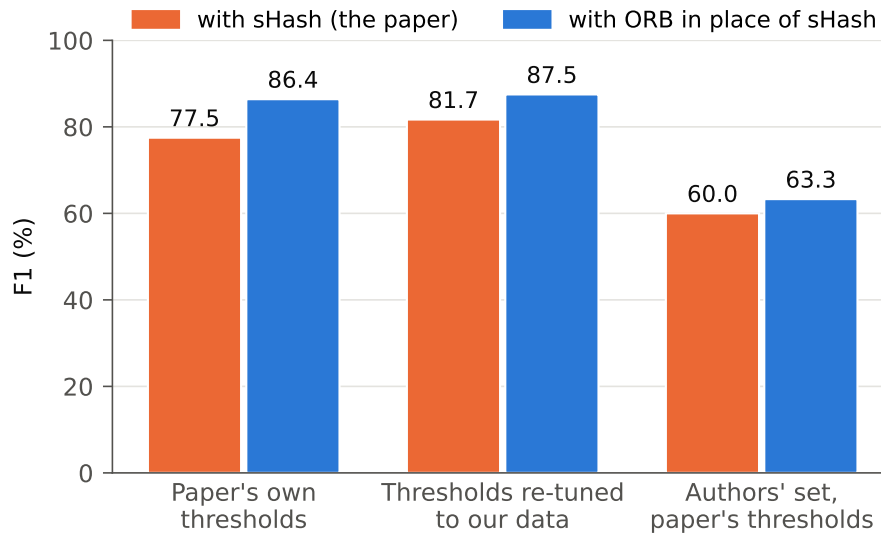


Figure 9: Result 3. F1 of the full two-of-four detector with sHash and with ORB, at the paper’s own thresholds, at thresholds re-tuned to our data, and on the authors’ reference set.

The authors’ reference set. On the 1,802 pairs the authors shared with us (202 copies, 1,600 manipulated non-copies, an 11.2% copy rate where flagging everything scores only 20.2), the paper’s detector at its own thresholds scores F1 60.0 (precision 55.2%, recall 65.8%). Swapping in ORB scores 63.3 (precision 48.0%, recall 93.1%), a modest **+3.3**. Here the swap trades precision for recall rather than holding precision equal, and the margin depends on ORB’s threshold: at the looser > 16 the swapped detector falls to 54.8 and loses. We therefore read it as a modest, threshold-sensitive confirmation on an independent generator, not as a headline result.

Comparison with the paper’s own numbers. We do not claim to beat the paper’s reported accuracy. Its tables report the 2-Minimal detector’s detection rates on duplicates and non-duplicates (for example 88.4% and 94.13% on DISC21) on a benchmark and search pipeline we did not reproduce. Those figures are not comparable to our pairwise precision, recall and F1.

8 Finding 2: What ORB Costs in Storage

The paper’s design stores every image’s hashes inside its mint transaction, so that detection is “fully self-contained within the blockchain”. That works for the four hashes. It does not work for ORB, and we report this as a finding.

At OpenCV’s default setting, ORB keeps about 455 descriptors of 32 bytes each, 14,560 bytes per image, the size of about 36 whole transactions. The default cap of 500 descriptors was never a deliberate choice, and it is not even reached. So we measured the whole accuracy-versus-size curve, giving every descriptor budget its own best threshold so that only the shape of the curve is compared (Figure 10). This protocol checks out: at the default budget it gives F1 90.8, within 0.2 of the train-tuned 90.6.

Representation	Bytes per image	Transactions (400 B each)
aHash + pHash + hsvHash	24 (fixed)	0.06
sHash	32 (median)	0.08
All four paper hashes	≈ 56	0.14
ORB, default (about 455 landmarks $\times$ 32 B)	14,560	36.4

Table 4: Measured storage per image, on 60 real originals.

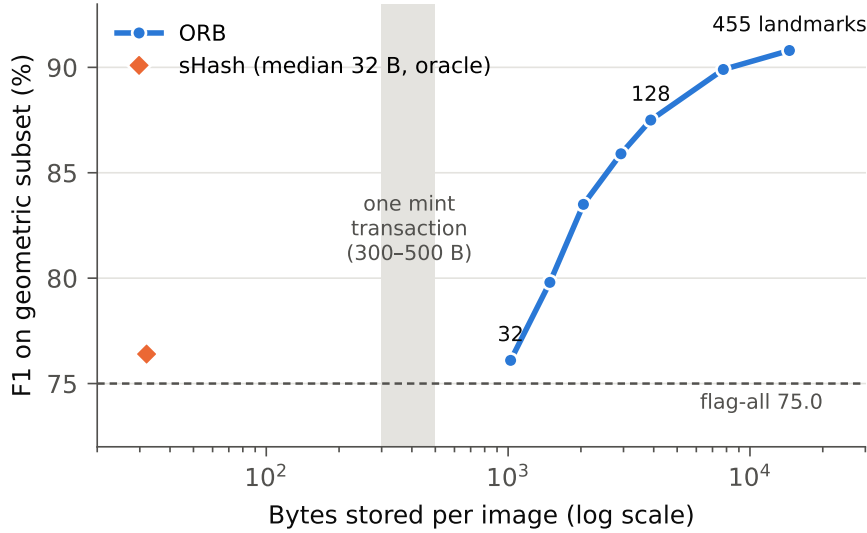


Figure 10: ORB accuracy on the geometric subset against bytes stored per image. The shaded band is one mint transaction. No point on the ORB curve lies inside it.

- **128 landmarks is the practical sweet spot:** 3,888 bytes (about 10 transactions) for F1 87.5, still far above sHash. As the budget shrinks, precision rises and recall falls; ORB becomes a stricter matcher.
- **32 landmarks is the floor:** 1,024 bytes, still 2.6 transactions, and F1 76.1, which only ties sHash (76.4) at 32 times its size. Below this, the best threshold collapses to “flag everything”.

Conclusion. There is no size at which ORB both fits in a transaction and beats sHash. The limit is architectural, not a matter of tuning: a signal made of a set of local features cannot travel in transaction metadata. The descriptor type also decides the index: one fixed-width string fits a BK-tree, while a set of binary descriptors needs LSH. The signal and its storage cannot be chosen separately. Section 11 discusses ways around the limit.

9 Finding 3: The Edit Classifier Does Not Improve the Detector

9.1 The classifier is accurate

On the held-out test split (11,991 query images), the edit classifier’s two decisions are:

A *dangerous miss* is calling a pixelated image “detail intact”, which would trust ORB exactly where it fails. This never happens: all 1,800 pixelated test images are recognised, including the 24×24 CryptoPunks where ORB itself is weakest. Raw eight-way accuracy is lower (89.2%), almost entirely because an exact copy and an untouched image are identical pixel for pixel and

Landmarks	Bytes	Transactions	Precision	Recall	F1
455 (default)	14,560	36.4	90.8%	90.8%	90.8%
242	7,760	19.4	94.2%	86.1%	89.9%
122 (cap 128)	3,888	9.7	96.1%	80.2%	87.5%
64	2,048	5.1	94.8%	74.7%	83.5%
32	1,024	2.6	90.8%	65.5%	76.1%
sHash (oracle)	32	0.08	66.5%	89.8%	76.4%

Table 5: Selected points of the size curve (geometric subset, best threshold per budget).

Decision	Precision	Recall	F1	Dangerous misses
Detail destroyed? ($\Rightarrow$ distrust ORB)	100%	100%	100%	0.0%
Colour changed? ($\Rightarrow$ distrust hsvHash)	98.1%	97.0%	97.5%	3.0%

Table 6: Edit classifier decisions on the held-out test split.

cannot be told apart from the query alone. Both mean “trust every signal”, so the confusion costs nothing.

9.2 But broken signals are silent, not wrong

Before using the classifier we measured, rather than assumed, which signal each edit breaks. Each signal was held at a fixed operating point flagging at most 10% of non-copies, and for each edit we recorded both how many copies it catches and how often it fires on the *wrong* original:

Edit	aHash caught / wrong	pHash caught / wrong	hsvHash caught / wrong	ORB caught
Flip / rotate	5.0 / 0.4	2.8 / 0.4	77.8 / 7.3	81.7
Crop / reposition	26.1 / 1.8	38.7 / 1.7	77.8 / 7.4	98.3
Background colour	60.7 / 7.7	82.7 / 7.5	16.1 / 3.8	98.9
Colour swap / saturate	75.2 / 6.1	78.2 / 6.0	28.3 / 6.7	79.8
Pixelated	55.3 / 1.9	65.5 / 2.7	79.3 / 8.2	28.5
Text / logo	93.4 / 5.0	96.7 / 5.7	93.1 / 7.8	100
Untouched non-copies (false alarms)	8.2	8.1	9.2	13.0

Table 7: Per-signal reliability map (%). “Caught” is the share of that edit’s copies the signal votes for; “wrong” is how often it votes for a deliberately wrong original.

Where a signal’s detection collapses, its false alarms collapse with it. aHash catches only 5.0% of rotated copies, and it fires on the wrong original only 0.4% of the time. hsvHash on background changes catches 16.1% and misfires 3.8%, below its normal 9.2%. A broken signal here does not vote randomly. It abstains. In a two-vote rule, telling the detector to ignore a signal that is already silent changes nothing.

9.3 Measured end to end

We compared the detector with ORB, with and without the classifier, at the same operating points as Table 3:

- static two-of-four with ORB: F1 87.5 (precision 97.8%, recall 79.2%);

- classifier-guided, no fallback: F1 87.4;
- classifier-guided with the static fallback: F1 87.5, identical to static.

The swap accounts for the whole gain; the classifier adds nothing measurable on top of it.

The classifier could only help through the other lever: lowering the quorum. When two signals are distrusted at once, the vote falls to “one of the remaining two”. With one edit per image, the classifier almost never distrusts two signals at once. A simulated third flag shows the lever is real. Distrusting aHash and pHash on rotated images leaves hsvHash and ORB and lowers the quorum to one, which raises flip/rotate recall from 58.4% to 97.7% at an unchanged 8.5% false-alarm rate. But that flag’s trigger reads transparent corners left by our generator’s rotation, and it scores 0% on the authors’ images. It cannot be relied on, so it is not counted as a result.

9.4 The classifier is specific to our generator

On the authors’ reference set the classifier degrades badly. Their CryptoPunks are already enlarged RGB images, ours are native 24×24 RGBA, so the transparency and histogram “combing” cues it relies on disappear. It then over-predicts pixelation, and the colour decision drops to 0% recall. Over-predicting pixelation only costs recall, because it makes the detector ignore ORB more often; it never makes the detector trust ORB wrongly. The classifier’s accuracy above is therefore a result within our own generator, not evidence that it recognises edits in general.

9.5 The vote rule depends on how common copies are

Sweeping the quorum on our test set suggests that “one of four” beats the paper’s “two of four” (F1 94.3 against 87.5). That is an artifact of a test set that is 82.6% copies. Reweighting the same results to a realistic share of copies reverses it (Figure 11): at 2%, 10% and 25% copies, two of four is better. Real copymints are rare among all mints, so the paper’s rule is the right one.

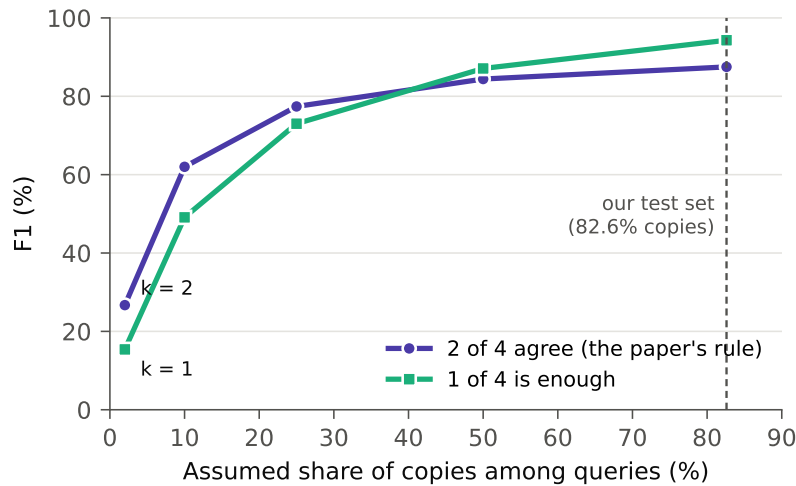


Figure 11: F1 of the ORB detector under the two vote rules, reweighted to an assumed share of copies. The paper’s two-of-four rule wins wherever copies are uncommon.

9.6 Phase E: two edits on one image

If the classifier fails only because single-edit data never breaks two signals at once, then images with two edits should change the verdict. We generated every ordered pair of the six non-exact edits (30 combinations; 18,000 copies and 2,400 non-copies, 88.2% copies). This test used the original panel with sHash, which stores nothing extra. The classifier would distrust sHash on pixelation (where sHash drops from 74.3% on crops to 38.1% while staying silent) and hsvHash on a colour change, lowering the quorum to one of two.

Detector (two-edit test set)	Precision	Recall	F1	$\Delta F1$
Paper’s panel (with sHash), static	96.7%	36.4%	52.9%	—
Same panel, classifier-guided	96.7%	36.4%	52.9%	+0.01
ORB in place of sHash, static	97.3%	45.1%	61.6%	+8.7
Flag everything	88.2%	100%	93.8%	

Table 8: Phase E results. Precision is about equal across detectors, so recall is the fair comparison.

The verdict did not change (Table 8). The reason is physical: **pixelation erases the colour trace**. To test whether the colour decision was merely being crowded out by pixelation in the eight-way classifier, we trained two independent yes/no detectors, one for destroyed detail and one for a colour change. The colour detector fires on 76–100% of colour edits combined with any non-pixelation edit, and on 0% of pixelation combined with a colour edit, in both orders. Pixelation collapses the palette, so the histogram combing the detector reads is gone. The one situation where the classifier could lower the quorum, detail destroyed *and* colour changed, cannot be observed with these cues. Without the static fallback the classifier even hurts (F1 50.3), because it drops sHash’s occasional correct vote without ever reaching the quorum change. Meanwhile the ORB swap still pays, at +8.7 F1.

This conclusion is limited to our generator and our pixelation strength, which is severe (down to 4–12% of the original size and back). Milder pixelation might leave a colour trace. We did not test that.

10 Discussion

10.1 What we learned

1. **Check the mathematics before optimising it.** We set out to make a search faster and found that the search itself was unsound. Speed is worth nothing until the structure returns correct answers.
2. **How a signal is stored decides how it can be searched.** One fixed-width string fits a BK-tree and a transaction field. A set of landmarks needs a different index and far more space. The signal and its storage cannot be chosen separately, and that is what the 14.5 KB figure really says.
3. **An accurate component is not the same as a useful one.** The edit classifier is excellent at its own task and changes nothing end to end, because the vote it feeds is unaffected by removing a signal that was already silent. A negative result is still a result, and it became informative once we could explain it.
4. **Reproduce the code, not only the text.** Library defaults, preprocessing, numeric precision and the direction of a comparison are part of the experiment. Where the paper’s description and its code differed, the code produced the numbers.

5. **Treat the evaluation as part of the engineering.** Several numbers that looked reasonable turned out to be wrong:

- an ORB baseline decided by gallery membership;
- an sHash threshold that, chosen to maximise F1 on a copy-heavy set, became “flag everything”;
- an ORB vote that was looser than the other three signals;
- a label-parsing bug that made every row a copy.

Each was caught by comparing against the flag-everything level, checking the class balance, or noticing that a threshold sweep landed at the edge of its range.

10.2 Difficulties

- **Exact reproduction.** Bit-exact parity with `imagehash` required reading Pillow’s C source to reproduce its mixed float/double arithmetic.
- **Silent failures.** ORB returns nothing below about 31 pixels, and the original pipeline dropped those images without a warning. Nothing crashed; an entire collection simply vanished from the evaluation.
- **A copy-heavy test set.** Because 82.6% of test pairs are copies, maximising F1 quietly rewards permissive detectors. We had to compare at equal precision and reweight to realistic copy rates to draw fair conclusions.
- **Our own generator.** Our labelled copies come from one PIL-based generator. The classifier partly learned how our generator works rather than what an edit looks like, which the authors’ independent images exposed.

11 Limitations and Future Work

Getting around ORB’s storage cost. We analysed three options; none was built:

- **Store descriptors off-chain with an on-chain commitment** (our preferred option): the transaction keeps a small hash of the descriptor set, and validators fetch the descriptors when needed. It gives up the paper’s self-contained design.
- **Aggregate the descriptors into one fixed-size vector** (VLAD or bag of visual words). It fits the budget but loses the RANSAC geometric check that makes ORB accurate.
- **Recompute descriptors on demand** from the image. Nothing is stored, but this gives up the speed that is the reason the paper stores hashes on-chain in the first place.

We did test the remaining storage-free route, a classifier-guided detector over the paper’s four cheap hashes, in Phase E: it gained +0.01 F1. A two-stage design, in which the cheap on-chain hashes shortlist candidates and off-chain ORB verifies them, is an untested idea for future work.

Pixelation. ORB’s one clean failure. The paper’s coarse hashes already cover it (pHash 65.5%, hsvHash 79.3%), but a structure signal that survives pixelation would close the gap. Our pixelation is also severe; the Phase E conclusion should be repeated across a range of pixelation strengths.

Tiny images. 24×24 pixel art is genuinely hard for any local-feature method. A detector could recognise low-resolution images and choose a representation suited to them.

A classifier that generalises. The classifier’s colour decision is specific to our generator and fails on the authors’ images. Training on several generators, resolutions and image formats, and dropping generator-specific features such as transparent corners, would be the first step. A more direct design would predict each signal’s reliability rather than an edit label.

Search at scale. All accuracy numbers are pairwise. A deployed ORB path also needs the LSH index and candidate verification to be benchmarked for retrieval recall, latency and memory.

A shared benchmark. The authors’ set is valuable but small and CryptoPunk-only. A public benchmark with several collections, separate rotation and mirror labels, combined edits and manipulated non-copies would make results like ours easier to compare.

12 Conclusion

We started from the copymint detector of Kotzer et al., in which four perceptual hashes vote and two agreeing votes mean a copy, and we set out to make it faster. Reimplementing it showed that one of the four signals, sHash, compares images one way and breaks the triangle inequality, so the BK-tree that searches it can silently skip a real copy.

We replaced sHash with ORB feature matching and changed nothing else. On the geometric edits sHash was built for, ORB scores F1 90.6 against sHash’s 76.4, even with sHash given a threshold chosen on the test set; at equal precision it catches 90.0% of copies against 27.2%. In the full detector at the paper’s own thresholds, the swap raises F1 from 77.5 to 86.4 at the same 98.1% precision, catching about 1,500 more of 11,400 copies. The gain holds when the paper’s thresholds are re-tuned (+5.8), on images with two edits (+8.7), and modestly on the authors’ own images (+3.3).

The improvement has a measured price. ORB needs about 14.5KB per image where the paper’s four hashes need about 56 bytes, and no descriptor budget lets it both fit in a mint transaction and beat sHash. Using ORB means changing where features are stored.

Our edit classifier predicts accurately which signals an edit breaks, yet it does not improve the detector, because broken signals go silent rather than voting wrongly. The one situation where it could help, detail destroyed and colour changed together, is not observable with our cues because pixelation erases the colour trace.

In short: we replaced one signal, measured it fairly, and reported its cost.

References

- [1] A. Kotzer, P. Reviriego, J. Conde Diaz, and O. Rottenstreich, “Combating NFT Copymints in Blockchain Networks: An Image Hashing Approach,” manuscript, with open-source implementation: <https://github.com/nft-duplication-detection/NFT-Duplications-detection-using-Image-hash-functions>, 2025.
- [2] A. Kotzer, M. Naamneh, O. Rottenstreich, and P. Reviriego, “Detection of NFT Duplications with Image Hash Functions,” in *IEEE International Workshop on Cryptocurrency Exchanges (CryptoEx)*, 2024.
- [3] J. Buchner, “ImageHash: A Python Perceptual Image Hashing Module,” <https://github.com/JohannesBuchner/imagehash>.
- [4] W. A. Burkhard and R. M. Keller, “Some Approaches to Best-Match File Searching,” *Communications of the ACM*, vol. 16, no. 4, pp. 230–236, 1973.
- [5] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An Efficient Alternative to SIFT or SURF,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, pp. 2564–2571, 2011.
- [6] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [7] A. Gionis, P. Indyk, and R. Motwani, “Similarity Search in High Dimensions via Hashing,” in *Proc. 25th International Conference on Very Large Data Bases (VLDB)*, pp. 518–529, 1999.
- [8] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [9] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [11] S. Mak, “NFT Classifier” dataset, Kaggle, <https://www.kaggle.com/datasets/shaunmak/nft-classifier>.
- [12] “CryptoPunks” dataset, Kaggle, <https://www.kaggle.com/datasets/tunguz/cryptopunks>.

A Setup and Run Guide

Everything in this report can be reproduced from the project repository. The datasets and score caches are not tracked in git (they are large and regenerable), so a fresh checkout starts by rebuilding them. Each script’s header comment documents its options.

A.1 Environment

Python 3.12 with the pinned dependencies in `training/requirements.txt` (Pillow, NumPy, ImageHash, SciPy, OpenCV headless, scikit-learn [10, 9]). The versions are pinned because a change in a resampling filter would silently change what “matching the paper” means.

```
python3 -m venv training/.venv
training/.venv/bin/pip install -r training/requirements.txt
```

Below, `py` stands for `training/.venv/bin/python`. Evaluation scripts default to the test split.

A.2 Data

1. Download the Kaggle `shaunmak/nft-classifier` set and flatten its 3,000 images into `data/raw/` as `azuki_#<id>.png`, `bayc_#<id>.png` and `cp_#<id>.png`. A two-image example of the layout is tracked in `data/example/`.
2. Generate the labelled copies. The split is by base image, 80/20, seed 42:

```
py training/generate_dataset.py          # data/train, data/test
py training/generate_dataset.py --mode compose # data/multi_{train,test}
```

A.3 Scoring (cached, run once per split)

```
# run each line with --split train, then --split test
py python/geometric/score_dataset.py --workers 8 # ORB inliers
py python/geometric/shash_baseline.py --workers 8 # sHash distances
py python/router/hash_reliability.py          # other hashes, reliability
py python/router/extract_features.py --workers 8 # 93 classifier features
```

Parallel runs (`-workers`) were checked to give the same output as serial ones.

A.4 Training and evaluation

```
py python/geometric/tune.py --split train # ORB threshold (> 16)
py python/router/train_router.py          # edit classifier
py python/router/train_multilabel.py      # independent heads (Phase E)

py python/geometric/compare_shash.py      # Result 1, Table 2
py python/geometric/evaluate.py --threshold 16 # Result 2
py python/geometric/diagnose_flip.py      # mirror vs rotation
py python/geometric/descriptor_budget.py  # storage, Table 5
py python/router/evaluate_router.py       # classifier, Table 6
py python/detector/evaluate_routed.py     # with/without classifier
py python/detector/evaluate_deployed.py   # Result 3, Table 3
py python/detector/evaluate_multi.py      # Phase E, Table 8
py python/detector/evaluate_reference.py  # authors' reference set
```

```
python3 report/figures/make_figures.py    # every chart in this report
```

The authors’ reference set is not distributed with the repository; the last detector command needs it in `data/reference/test_manipulations/`. `python/geometric/verify_baseline.py` reproduces the retired baseline’s confusion matrix; it reads that pipeline’s original gallery files, passed with `-project`.

A.5 C11 hash suite

The bit-exact C11 reimplementation in `src/hashes/` builds with CMake:

```
cmake -S . -B build && cmake --build build
```

B Operating Points

Signal	Paper’s own thresholds	Our detector ($\leq 10\%$ false alarms on train)	Signal study (geometric subset)
aHash	≤ 7	≤ 7	—
pHash	≤ 15	≤ 19	—
hsvHash	≤ 3	≤ 3	—
sHash	≤ 17	≤ 21	≤ 28 (oracle, chosen on test)
ORB	—	> 23 inliers	> 16 inliers (chosen on train)

Table 9: Thresholds used in this report. Hash thresholds are Hamming distances; sHash’s is its averaged one-way distance.

C Data Notes

- The generated dataset comes from public Kaggle collections; the generator and a small example are in the repository.
- The authors’ reference set (202 manipulated CryptoPunks, 200 originals, 1,802 labelled pairs) was shared with us privately. It is kept locally and not redistributed.
- All accuracy figures are pairwise. No index or gallery is involved in any reported number.